

**Software de Gestión Administración Edificio Mirador
(DAS) Documento Arquitectura de Software
Versión 1.0**

Identificación de Documento

Identificación	DAS_SGGC_Edificio Mirador
Proyecto	Sistema de Gestión de Gastos Comunes - Edificio Mirador
Versión	1.0

Documento mantenido por	Grupo 1: Pamela Acuña Gabriel Martínez Daniel Castro
Fecha de última revisión	05-04-2026
Fecha de próxima revisión	12-04-2026

Documento aprobado por	
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
25-03-2026	0.1	Creación de la estructura base y redacción de Cap. 1.	Pamela Acuña
28-03-2026	0.5	Incorporación de Requerimientos Funcionales y No Funcionales.	Daniel Castro
03-04-2026	1.0	Corrección de la Arquitectura Monolítica y versión final para entrega.	Gabriel Martínez

TABLA DE CONTENIDOS

1. INTRODUCCIÓN	5
1.1. Contexto del Problema	5
1.2. Propósito	5
1.3. Ámbito	6
1.4. Definiciones, acrónimos y abreviaciones	6
1.5. Resumen ejecutivo	6
1.6. Arquitectura del sistema	7
2. VISIÓN DEL SISTEMA	7
2.1. Descripción general del sistema	7
2.2. Objetivos del sistema	8
2.3. Requerimientos funcionales	9
2.4. Requerimientos no funcionales	11
2.5. Supuestos y dependencias	12
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	14
3.2. Justificación del estilo según el contexto del sistema	14
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	14
4.1. VISTA DE ESCENARIO	14
4.1.1. Propósito	14
4.1.2. Actores	14
4.1.3. Diagrama general de casos de uso	14
4.1.4. Diagrama de casos de uso específicos	14
4.1.6. Especificación de casos de uso	14
4.2. VISTA LÓGICA	14
4.2.1. Propósito	14
4.2.2. Diagrama de clases	14
4.2.3. Descripción diagrama de clases	14
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	14
4.3.1. Propósito	14
4.3.2. Diagrama de componente	14
4.3.3. Descripción diagrama de componente	14
4.3.4. Diagrama de paquete	14
4.3.5. Descripción diagrama de paquete	14
4.4. VISTA DE PROCESOS	14
4.4.1. Propósito	14
4.4.2. Diagrama de actividad	14
4.4.3. Descripción diagrama de actividad	14
4.5. VISTA FÍSICA	14
4.5.1. Propósito	14

4.5.2.	Diagrama de despliegue	14
4.5.3.	Descripción diagrama de despliegue	14
5.	REQUISITOS DE CALIDAD	14
5.1.	Propósito	14
5.3.	Reglas y criterios de evaluación de calidad	14
6.	PRINCIPIOS DE DISEÑO APLICADOS	14
6.1.	Propósito	14
6.2.	Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)	14
7.	PROTOTIPO	14
7.1.	Propósito	14
7.2.	Mockups (imágenes con una breve descripción)	14
7.3.	Justificar herramientas de prototipado	14
8.	EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	14
8.1.	Propósito	14
8.2.	Lista de verificación	14
8.3.	Análisis y métricas de resultados	14
9.	CONTROL DE VERSIONES	14
9.1.	Propósito	14
9.2.	Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)	14
9.3.	Justificar herramientas de versionamiento	14
10.	CONCLUSIONES	14
11.	BIBLIOGRAFÍA	14
12.	ANEXOS	14
12.1.	Carta Gantt	14

1. INTRODUCCIÓN

1.1 Contexto del problema

El Edificio Mirador, construido en el año 2010, es un complejo habitacional compuesto por 160 departamentos distribuidos en 20 pisos. Su estructura demográfica actual indica que un 60% de la ocupación corresponde a propietarios, mientras que el 40% restante se encuentra bajo régimen de arriendo. Esta distribución exige una gestión administrativa y financiera altamente precisa, dada la diversidad de actores involucrados en el pago de las obligaciones.

Actualmente, la problemática central radica en que la administración opera de manera íntegramente empírica y manual, careciendo de sistemas de software o registros electrónicos formalizados para su gestión diaria. Esta ausencia de digitalización y automatización ha desencadenado una serie de ineficiencias críticas a nivel operativo y de negocio:

Inestabilidad Financiera: Existe un deficiente control del flujo de caja, lo que impacta directamente en la liquidez y dificulta el cumplimiento oportuno de las obligaciones financieras del edificio.

Fallas Operativas: La gestión de cobros resulta ineficiente y se evidencian retrasos constantes en la ejecución de las mantenciones programadas de los espacios comunes.

Crisis de Confianza y Transparencia: La dependencia exclusiva de procesos de cálculo y registro manuales ha incrementado el margen de error. Esto se ha traducido en un alza significativa de los reclamos, mermando gravemente la confianza de los residentes (tanto propietarios como arrendatarios) hacia la administración.

1.2. Propósito

El propósito del presente documento es definir, estructurar y comunicar la arquitectura de software para el nuevo **Sistema de Gestión de Gastos Comunes (SGGC)** del Edificio Mirador.

A nivel de negocio y sistema, el propósito fundamental de la solución tecnológica es digitalizar y automatizar el ciclo completo de la administración financiera del edificio. Esto abarca desde el cálculo exacto y prorrateo de gastos comunes —basado en la superficie en metros cuadrados (M2) de cada unidad—, hasta el registro de pagos, la emisión de comprobantes y el control activo de la morosidad.

Mediante la implementación de este sistema, se busca erradicar los errores operativos derivados de la digitación manual, optimizar los tiempos de gestión de la administración y, como objetivo estratégico, restaurar la transparencia y la confianza de la comunidad de residentes al proporcionarles acceso claro y oportuno a su información financiera.

1.3. Ámbito

El ámbito de este proyecto comprende el diseño, desarrollo e implementación del **Sistema de Gestión de Gastos Comunes (SGGC)** para el Edificio Mirador. El sistema centralizará la información y automatizará los flujos de trabajo administrativos.

1.3.1. Alcance del Sistema (Dentro del Ámbito): El desarrollo contempla la entrega de los siguientes módulos funcionales principales:

- **Módulo de Gestión de Usuarios y Propiedades:** Registro y mantención (CRUD) de la

información de copropietarios, arrendatarios, personal administrativo y el maestro de departamentos (incluyendo la superficie en M2 de cada unidad).

- **Módulo de Finanzas y Prorratio:** Motor de cálculo automatizado para la distribución de gastos comunes y extraordinarios, aplicando reglas de negocio basadas en el prorratio por metro cuadrado.
- **Módulo de Pagos y Morosidad:** Registro de pagos recibidos, emisión de comprobantes de pago, cálculo de deuda acumulada (intereses/multas) y generación de reportes de morosidad.
- **Módulo de Comunicación:** Canal digital para la emisión de notificaciones automáticas de cobro y una bandeja de entrada para la trazabilidad de solicitudes o reclamos de los residentes.

1.3.2. Exclusiones (Fuera del Ámbito): Para garantizar la viabilidad y el enfoque de esta primera versión del sistema, se excluyen explícitamente las siguientes funcionalidades:

- **Integraciones Bancarias Directas:** El sistema no se integrará vía API con pasarelas de pago (ej. Transbank, Webpay) ni con cuentas bancarias para la conciliación automática. Los pagos serán reportados y validados administrativamente.
- **Gestión de Recursos Humanos:** No se incluirá un módulo para el cálculo de remuneraciones, pago de cotizaciones o control de asistencia del personal del edificio (conserjes, aseo).
- **Control de Acceso e IoT:** El sistema no interactuará con hardware del edificio, como barreras de estacionamiento, cámaras de seguridad o controles de acceso biométrico.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
DAS	Documento de Arquitectura de Software. Es el presente documento, que proporciona una visión arquitectónica global del sistema.
SGGC	Sistema de Gestión de Gastos Comunes. Nombre asignado a la solución de software que se construirá para el Edificio Mirador.
CRUD	<i>Create, Read, Update, Delete</i> (Crear, Leer, Actualizar, Eliminar). Acrónimo técnico que hace referencia a las cuatro operaciones básicas de gestión de datos en el sistema.
MVC	<i>Model-View-Controller</i> (Modelo-Vista-Controlador). Patrón de diseño de software que separa la aplicación en tres componentes principales para facilitar su desarrollo y mantenimiento.

1.5. Resumen ejecutivo

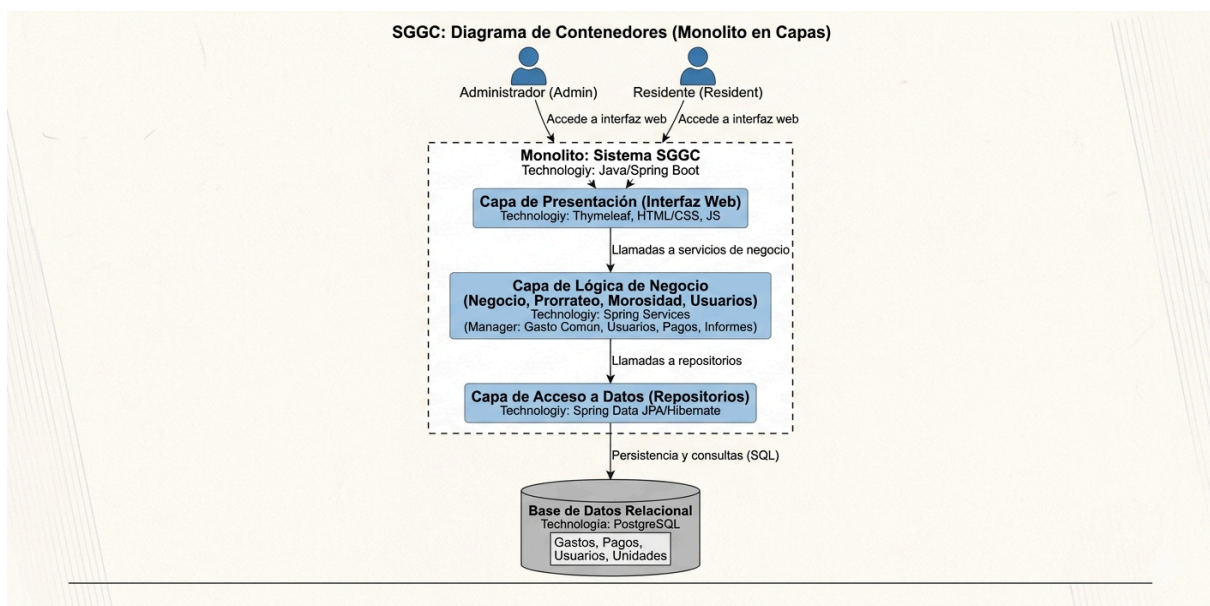
El presente Documento de Arquitectura de Software (DAS) describe la solución tecnológica propuesta para resolver las deficiencias administrativas y operativas del Edificio Mirador. Ante la ineficiencia,

falta de transparencia y los errores derivados de la actual gestión manual, se propone el desarrollo del **Sistema de Gestión de Gastos Comunes (SGGC)**. Esta plataforma digital está orientada a automatizar y transparentar el ciclo financiero completo del recinto habitacional.

A nivel técnico, el sistema ha sido diseñado bajo un enfoque de **Arquitectura Monolítica** estructurada en tres capas lógicas (Presentación, Lógica de Negocio y Datos) y apoyada en el patrón de diseño MVC. Esta decisión arquitectónica se justifica por la escala del proyecto (160 unidades y un volumen acotado de usuarios concurrentes), garantizando un desarrollo ágil, un despliegue unificado y un mantenimiento simplificado, evitando la sobreingeniería de sistemas distribuidos.

La implementación del SGGC aportará un valor inmediato al negocio al automatizar el cálculo de prorrateo exacto basado en metros cuadrados (M2), centralizar el registro de pagos y establecer un control proactivo de la morosidad. En consecuencia, esta transformación digital permitirá estabilizar el flujo de caja, erradicar los errores de digitación y, como beneficio principal, restaurar la confianza de la comunidad de propietarios y arrendatarios mediante el acceso a información financiera clara, auditable y en tiempo real.

1.6. Arquitectura del sistema



2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El **Sistema de Gestión de Gastos Comunes (SGGC)** es una plataforma web centralizada diseñada para la administración integral del Edificio Mirador. Su función principal es digitalizar y orquestar los procesos financieros y operativos que actualmente se realizan de forma manual, estableciendo una única fuente de verdad para la administración, el comité y los residentes.

Desde una perspectiva operativa, el sistema se estructura en torno a cuatro ejes funcionales principales:

- **Gestión de Propiedades y Usuarios:** Actúa como el directorio maestro del edificio. Permite el registro detallado de los 160 departamentos, almacenando de forma inmutable la superficie útil en metros cuadrados (m^2) de cada unidad, y vinculando dinámicamente a los residentes actuales (propietarios o arrendatarios) con sus respectivas propiedades.
- **Motor de Prorrrateo y Finanzas:** Es el núcleo transaccional del sistema. A partir del registro de los gastos totales del mes (sueldos, servicios, mantenciones), el sistema automatiza el cálculo matemático para distribuir el cobro a cada departamento, basándose estrictamente en el prorrrateo por m^2 . Esto elimina los errores humanos de cálculo y digitación.
- **Control de Recaudación y Morosidad:** Permite a la administración registrar los abonos y pagos realizados por los residentes. Paralelamente, el sistema cuenta con rutinas automáticas que evalúan el estado de cuenta de cada departamento, detectando cuotas impagas, calculando intereses o multas, y actualizando el estado de "Morosidad" en tiempo real.
- **Portal de Transparencia y Comunicación:** Proporciona una interfaz dedicada para que los residentes puedan consultar sus estados de cuenta históricos, descargar sus comprobantes de pago y visualizar el desglose exacto de los gastos del edificio, fomentando la confianza y reduciendo la carga de consultas directas hacia la administración.

En conjunto, el SGGC transforma un ecosistema administrativo propenso al error y la desinformación en una operación automatizada, auditable y transparente.

2.2. Objetivos del sistema

2.2.1. Objetivo General

Desarrollar e implementar un **Sistema de Gestión de Gastos Comunes (SGGC)** centralizado y automatizado, que permita digitalizar la administración financiera y operativa del Edificio Mirador, garantizando exactitud en los cobros, eficiencia en la gestión de morosidad y total transparencia en la información entregada a la comunidad de residentes.

2.2.2. Objetivos Específicos

Para alcanzar el objetivo general, el sistema deberá cumplir con los siguientes objetivos específicos:

- **Automatizar el Prorrrateo:** Calcular de forma automática y exacta los gastos comunes mensuales de cada departamento, utilizando como base matemática la superficie útil en metros cuadrados (m^2) de cada unidad, eliminando los errores de cálculo manual.
- **Centralizar la Información:** Crear un registro digital único y actualizado que vincule a los 160 departamentos con sus respectivos residentes (propietarios o arrendatarios) y su historial de pagos.
- **Controlar y Reducir la Morosidad:** Implementar rutinas de seguimiento que identifiquen automáticamente las cuentas impagas, calculen los recargos correspondientes y emitan notificaciones de cobro oportunas a los deudores.
- **Habilitar la Autogestión y Transparencia:** Proveer a los residentes de una interfaz web intuitiva donde puedan consultar su estado de cuenta en tiempo real, visualizar el desglose de los gastos del edificio y descargar sus comprobantes de pago.

- **Optimizar los Tiempos Administrativos:** Reducir significativamente las horas-hombre que la administración dedica actualmente a la digitación de datos, consolidación de planillas y atención de consultas rutinarias sobre saldos adeudados.

2.3. Requerimientos funcionales

A continuación, se detallan todas las funcionalidades que el sistema debe proveer para satisfacer los objetivos del negocio. Se han redactado sin ambigüedades, especificando acciones exactas para garantizar su correcta interpretación e implementación.

Módulo 1: Gestión de Usuarios y Propiedades

ID	Nombre de Funcionalidad	Descripción Específica y Reglas de Negocio
RF-01	Registro de Departamentos	El sistema debe permitir al Administrador registrar, modificar y visualizar los 160 departamentos del edificio. Es obligatorio registrar el número de unidad y su superficie útil exacta en metros cuadrados (m2). Este valor numérico será la base inmutable para cálculos futuros.
RF-02	Registro de Residentes	El sistema debe permitir al Administrador crear perfiles de residentes, exigiendo: Nombre, RUT, Correo Electrónico y Teléfono. Además, debe asignar obligatoriamente una
RF-03	Vinculación Residente-Propiedad	El sistema debe permitir asignar uno o más residentes a un departamento específico. Solo los residentes vinculados a un departamento activo podrán visualizar su información financiera
RF-04	Control de Acceso y Roles	El sistema debe autenticar a los usuarios mediante Correo y Contraseña. Debe aplicar permisos estrictos según dos roles: Administrador (acceso total a configuración y finanzas globales) y Residente (acceso de solo lectura a su departamento y cobros personales).

Módulo 2: Finanzas y Prorratio (Gastos Comunes)

ID	Nombre de Funcionalidad	Descripción Específica y Reglas de Negocio
RF-05	Registro de Egresos Mensuales	El sistema debe permitir al Administrador ingresar los gastos del edificio de un mes específico (ej. luz, agua común, sueldos, mantenciones de ascensores), registrando monto, fecha, categoría y subiendo un archivo de respaldo (boleta/factura).

RF-06	Cálculo de Prorratio por M2	El sistema debe calcular automáticamente el gasto común de cada departamento aplicando la siguiente fórmula matemática obligatoria: $(\text{Total de Egresos del Mes}) \times (\text{Metros Cuadrados del Departamento} / \text{Total de Metros Cuadrados del Edificio})$.
RF-07	Generación de Estado de Cuenta	Una vez ejecutado el prorratio, el sistema debe generar un documento digital (PDF) por cada departamento que detalle: Mes de cobro, desglose de gastos del edificio, factor de prorratio (M2), saldo anterior adeudado y total a pagar.

Módulo 3: Pagos y Morosidad

ID	Nombre de Funcionalidad	Descripción Específica y Reglas de Negocio
RF-08	Registro Manual de Pagos	El sistema debe permitir al Administrador registrar un pago recibido, exigiendo ingresar obligatoriamente: Departamento que paga, Monto abonado, Fecha de transacción y Medio de pago (Transferencia, Efectivo, Cheque).
RF-09	Emisión de Comprobantes	Al registrar exitosamente un pago (RF-08), el sistema debe generar automáticamente un comprobante digital, rebajar la deuda del departamento y dejar el comprobante disponible en el portal del residente.
RF-10	Detección Automática de Morosidad	El sistema debe verificar automáticamente, el primer día hábil después de la fecha de vencimiento configurada, los saldos de todos los departamentos. Si un departamento tiene saldo pendiente mayor a \$0, el sistema debe cambiar su estado a "Moroso".
RF-11	Cálculo de Multas/Intereses	El sistema debe aplicar automáticamente un porcentaje de interés (configurable por el Administrador) al saldo vencido de los departamentos en estado "Moroso", sumándolo al Estado de Cuenta del mes siguiente.

Módulo 4: Comunicación y Transparencia

ID	Nombre de Funcionalidad	Descripción Específica y Reglas de Negocio
RF-12	Portal de Transparencia (Residente)	El sistema debe permitir al Residente visualizar un panel resumen con: Su deuda actual, el historial de sus pagos de los últimos 12 meses y el acceso de descarga a los Estados de Cuenta de su departamento.

2.4. Requerimientos no funcionales

Los siguientes requerimientos definen los atributos de calidad, restricciones técnicas y estándares de rendimiento que el sistema debe cumplir para garantizar una operación robusta y segura.

1. Seguridad y Privacidad

RNF-01	Cifrado en Tránsito	Toda la comunicación entre la capa de presentación (navegador web) y el servidor debe estar cifrada obligatoriamente mediante protocolo HTTPS (TLS 1.2 o superior) . No se permitirán conexiones HTTP en texto plano.
RNF-02	Protección de Credenciales	Las contraseñas de los usuarios (Administradores y Residentes) nunca deben almacenarse en texto plano en la base de datos. Deben ser encriptadas utilizando un algoritmo de hashing fuerte con "salt" (ej. Bcrypt o Argon2).
RNF-03	Cierre de Sesión Automático	Por motivos de seguridad financiera, el sistema debe invalidar la sesión activa de cualquier usuario tras 15 minutos de inactividad continua, obligando a un nuevo inicio de sesión.

2. Rendimiento y Eficiencia

RNF-04	Tiempo de Respuesta (UI)	El tiempo de carga de las vistas de la aplicación (ej. consultar el estado de cuenta) bajo condiciones normales de red no debe superar los 2 segundos en el 95% de las peticiones.
RNF-05	Tiempo de Procesamiento (Motor)	El algoritmo de la capa de lógica de negocio encargado del "Cálculo de Prorrates Mensual" para la totalidad de los 160 departamentos, debe completarse en un tiempo máximo de 5 segundos desde que el administrador confirma la acción.
RNF-06	Concurrencia	El sistema debe ser capaz de soportar hasta 50 usuarios concurrentes navegando o realizando transacciones

		simultáneamente sin degradar el tiempo de respuesta establecido en el RNF-04.
--	--	---

3. Usabilidad y Compatibilidad

RNF-07	Diseño Responsivo	La interfaz de usuario debe adaptarse dinámicamente a diferentes resoluciones de pantalla. Debe ser 100% funcional y legible en dispositivos Móviles (Smartphones), Tablets y Escritorio (PC/Laptop) .
RNF-08	Compatibilidad de Navegadores	El sistema no requerirá la instalación de software local. Debe ser totalmente compatible y renderizar correctamente en las dos últimas versiones estables de los navegadores: Google Chrome, Mozilla Firefox, Safari y Microsoft Edge .

4. Disponibilidad y Mantenibilidad

RNF-09	Disponibilidad (Uptime)	El sistema debe garantizar un nivel de disponibilidad operativa del 99.5% mensual (excluyendo ventanas de mantenimiento programadas y notificadas con 48 horas de anticipación).
RNF-10	Arquitectura y Código	El código fuente debe estar estructurado bajo el patrón MVC (Modelo-Vista-Controlador) dentro de la arquitectura monolítica. Todo servicio de backend debe contar con comentarios técnicos para facilitar su futura mantenibilidad por otros desarrolladores.

2.5. Supuestos y dependencias

Para la correcta planificación, desarrollo e implementación del **Sistema de Gestión de Gastos Comunes (SGGC)**, se han establecido las siguientes condiciones base. Cualquier alteración en estos puntos podría impactar el alcance, los plazos o la arquitectura del sistema.

2.5.1. Supuestos

- **Infraestructura del Usuario Final:** Se asume que tanto el personal administrativo como los residentes cuentan con dispositivos compatibles (computadoras, tablets o smartphones) y conexión estable a Internet para acceder a la plataforma web. El sistema no proveerá hardware.
- **Calidad y Disponibilidad de Datos Iniciales:** Se asume que, antes del paso a producción, la administración actual del Edificio Mirador entregará la "Data Maestra" depurada, estructurada y sin inconsistencias (ej. planilla Excel con el detalle exacto de los 160 departamentos, sus respectivos metros cuadrados (m2), datos de contacto de residentes

actuales y saldos históricos). El sistema no se hará responsable por cálculos erróneos derivados de datos iniciales corruptos.

- **Alfabetización Digital:** Se asume que el público objetivo posee competencias digitales básicas para la navegación web estándar (ingreso de credenciales, llenado de formularios, descarga de PDFs).

2.5.2. Dependencias Externas

- **Infraestructura de Alojamiento (Cloud/Hosting):** El funcionamiento en ambiente de producción depende estrictamente de la contratación, configuración y disponibilidad de un servidor web (ej. VPS o servicio Cloud) y de un motor de Base de Datos Relacional para alojar la arquitectura monolítica.
- **Servicio de Correos Transaccionales (SMTP):** Para dar cumplimiento al requerimiento funcional de notificaciones automáticas y alertas de cobro (RF-13), el sistema depende de la integración con un proveedor de servicios de correo electrónico externo (ej. SendGrid, Amazon SES o similar).
- **Aprobación del Comité de Administración:** El avance de las etapas críticas del desarrollo y el despliegue final dependen directamente de la disponibilidad del comité de administración para realizar las pruebas de aceptación de usuario (UAT) y validar que el motor de prorrateo funcione según sus reglas de negocio.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado (ej. monolítico, microservicios, SOA, capas)

3.2. Justificación del estilo según el contexto del sistema

3.3. Patrones de diseño aplicados (ej. patrón MVC, repositorio, etc.)

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

4.1.2. Actores

4.1.3. Diagrama general de casos de uso

4.1.4. Diagrama de casos de uso específicos

4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	Exportar saldos y puntos a vencer	
CU-002-001	Exportar actividades	
CU-002-002	Importar deuda vencida por PDV	
CU-004-001	Generación Archivo PDA Importación	

4.1.6. Especificación de casos de uso

Caso de Uso	[Nombre del Caso de Uso]	Identificador: [Del caso de uso]
Actores	[Listado de los actores que tienen participación en el caso de uso]	
Tipo	[Tipo de caso de uso, primario, secundario, opcional]	
Referencias	[Requerimientos o funcionalidades incluidas en este caso de uso. Casos de uso relacionados.]	
Precondición	[Condiciones sobre el estado del sistema que deben cumplirse para iniciar]	

	el caso de uso]
Postcondición	[Efectos inmediatos que tienen la ejecución del caso de uso sobre el estado del sistema]
Descripción	[Descripción del caso de uso]
Resumen	[Resumen de alto nivel del funcionamiento]

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
[Nro. de paso]	[Actor ejecutor o especifica si es el sistema o subsistema]	[Descripción del paso actividad ejecutado]
[Se describe el proceso o secuencia de pasos ejecutadas usando frases cortas] [Cada paso del proceso puede ser ejecutado por los Actores o por el sistema] [Se describe la secuencia de acciones realizadas por los actores y la secuencia de actividades realizada por el sistema como respuesta].		

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
[Número de paso]	[Descripción de la secuencia de acciones alternas para el número de actividad indicado. Debe hacer referencia al número de paso en el curso normal]
[Cada paso descrito en el curso normal, puede tener actividades alternas, según la distribución de escenarios que ocurra en el flujo de procesos, en esta ficha se completa para cada actividad (haciendo referencia a su número) las posibles secuencias alternas]	

4.2. VISTA LÓGICA

4.2.1. Propósito

4.2.2. Diagrama de clases

4.2.3. Descripción diagrama de clases

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

- 4.3.1. Propósito
- 4.3.2. Diagrama de componente
- 4.3.3. Descripción diagrama de componente
- 4.3.4. Diagrama de paquete
- 4.3.5. Descripción diagrama de paquete

4.4. VISTA DE PROCESOS

- 4.4.1. Propósito
- 4.4.2. Diagrama de actividad
- 4.4.3. Descripción diagrama de actividad

4.5. VISTA FÍSICA

- 4.5.1. Propósito
- 4.5.2. Diagrama de despliegue
- 4.5.3. Descripción diagrama de despliegue

5. REQUISITOS DE CALIDAD

5.1. Propósito

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN

5.3. Reglas y criterios de evaluación de calidad

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc.).

Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizad (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt

CARTA GANTT

[illegible]